

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»

Домашнее задание №5

**Реализация межсервисного взаимодействия посредством
очереди сообщений**

Выполнил:
студент группы К3339
Катков Алексей Сергеевич

Проверил:
Добряков Давид Ильич

Санкт-Петербург
2026 г.

1 Задача

Подключить и настроить RabbitMQ для организации обмена сообщениями между микросервисами. Реализовать межсервисное взаимодействие посредством брокера сообщений RabbitMQ.

2 Ход работы

В качестве брокера сообщений был выбран RabbitMQ. Для его запуска использовался Docker-контейнер на основе образа `rabbitmq:3-management`. После запуска брокер стал доступен для подключения микросервисов.

В рамках работы была доработана существующая микросервисная архитектура, состоящая из следующих сервисов:

- API Gateway;
- Auth Service;
- Recipe Service;
- Social Service.

Для реализации межсервисного взаимодействия были добавлены механизмы публикации и обработки сообщений через RabbitMQ.

Recipe Service выступает в роли производителя сообщений (Producer). После создания нового рецепта сервис формирует событие `recipe.created` и отправляет его в очередь RabbitMQ.

Социальный сервис выступает в роли потребителя сообщений (Consumer). Он подписывается на очередь RabbitMQ и ожидает поступления событий. После получения сообщения сервис выполняет его обработку и выводит информацию о событии в консоль приложения.

Для проверки работоспособности системы были запущены:

- RabbitMQ;
- API Gateway;
- Auth Service;
- Recipe Service;
- Social Service.

После запуска сервисов был выполнен POST-запрос на создание нового рецепта через API Gateway.

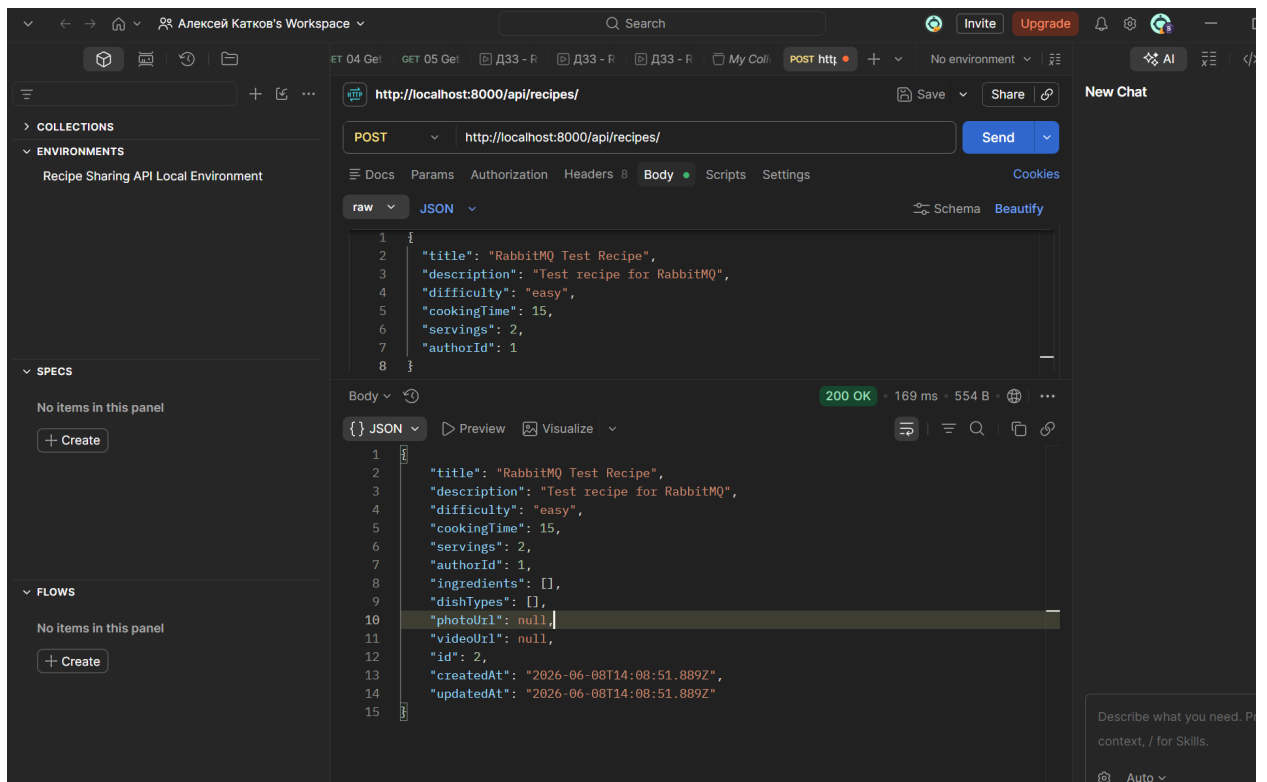


Рис. 1: Создание рецепта через API Gateway

В результате выполнения запроса был успешно создан новый рецепт и сохранён в базе данных.

После сохранения объекта сервис рецептов отправил событие `recipe.created` в RabbitMQ.

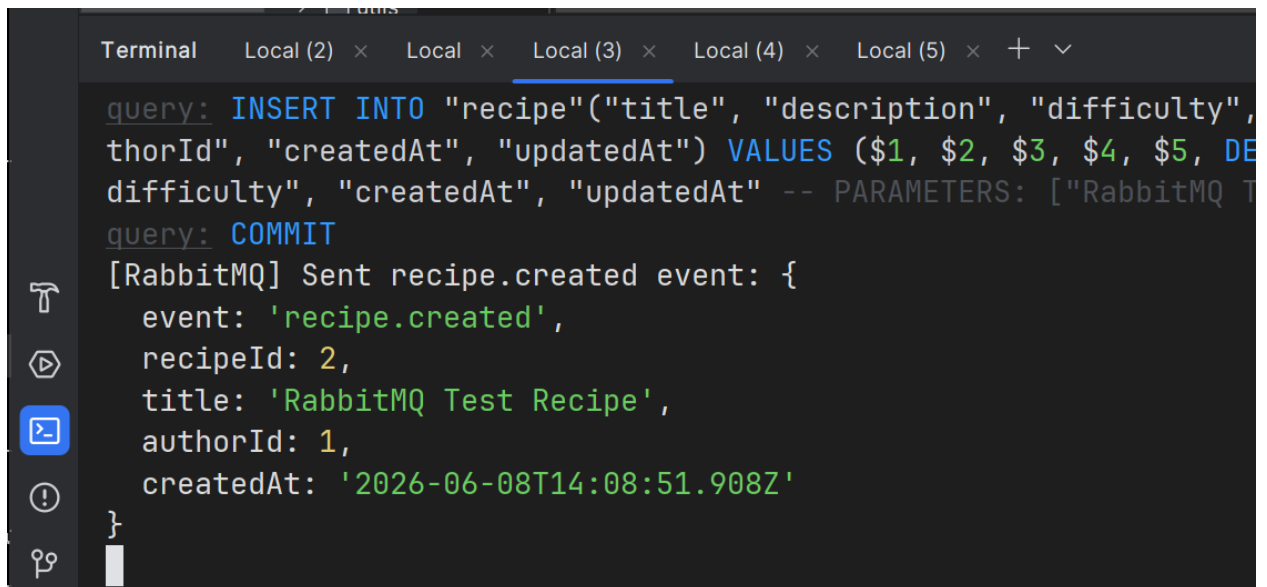


Рис. 2: Отправка события `recipe.created` в RabbitMQ

Социальный сервис получил сообщение из очереди и обработал его.

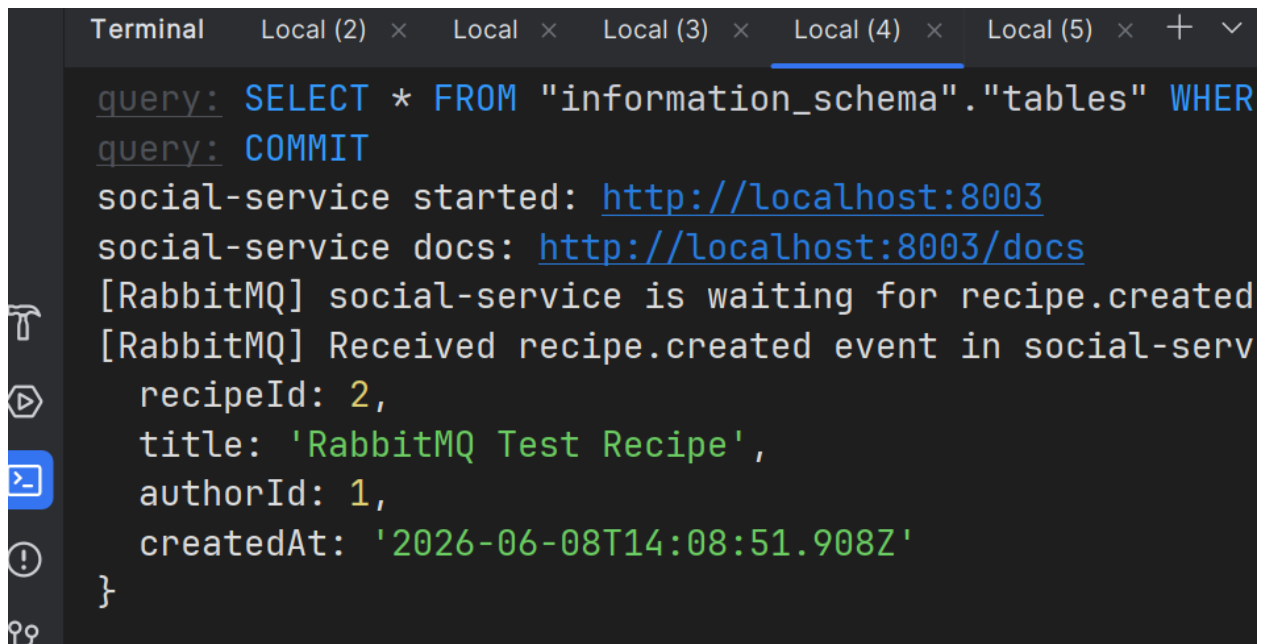
A screenshot of a terminal window with a dark background. The window has several tabs at the top: 'Terminal', 'Local (2)', 'Local', 'Local (3)', 'Local (4)', and 'Local (5)'. The 'Terminal' tab is active. The text in the terminal shows a SQL query: 'query: SELECT * FROM "information_schema"."tables" WHERE', followed by 'query: COMMIT'. Then, it shows 'social-service started: http://localhost:8003' and 'social-service docs: http://localhost:8003/docs'. Below that, it says '[RabbitMQ] social-service is waiting for recipe.created' and '[RabbitMQ] Received recipe.created event in social-serv'. Finally, it displays a JSON object: 'recipeId: 2,', 'title: 'RabbitMQ Test Recipe'', 'authorId: 1,', 'createdAt: '2026-06-08T14:08:51.908Z'', and a closing brace '}'.

Рис. 3: Получение события recipe.created сервисом social-service

В результате была реализована следующая схема взаимодействия микросервисов:

API Gateway → Recipe Service → RabbitMQ → Social Service

Таким образом сервисы взаимодействуют асинхронно и не зависят напрямую друг от друга.

3 Вывод

В ходе выполнения работы был изучен принцип межсервисного взаимодействия посредством брокера сообщений RabbitMQ.

Был выполнен запуск и настройка RabbitMQ, реализована публикация событий в сервисе рецептов и получение сообщений в социальном сервисе.

Проведённое тестирование подтвердило корректную работу обмена сообщениями между микросервисами. При создании рецепта событие успешно публикуется в очередь RabbitMQ и обрабатывается другим сервисом.

Поставленная задача выполнена в полном объёме.